INF201 - Deliverable 2

Campus Coffee Shop Sales Pipeline

This notebook discovers transaction logs in nested folders, reads mixed text encodings, removes system noise and price outliers, corrects NOK prices, and generates the required reports.

GitHub repository: https://github.com/fifkodz1/inf201-deliverable-2

In [1]:
```python
import json
import re
from pathlib import Path

import numpy as np
import pandas as pd

DATA_DIR = Path("data")
CONFIG_PATH = DATA_DIR / "store_config.json"
```

Task 1 - File ingestion and regular expressions

`read_text_file` first attempts UTF-8 and falls back to ISO-8859-1 when UTF-8 decoding fails. `parse_log` uses one regular expression to capture the six required transaction fields. Lines that do not match the complete pattern are treated as system noise and ignored.

In [2]:
```python
LOG_PATTERN = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] "
    r"\[(?P<register_id>[A-Za-z0-9_]+)\] "
    r"item=(?P<item>[A-Za-z][A-Za-z0-9 _-]*) "
    r"price=(?P<price>\d+(?:\.\d+)?) "
    r"(?P<currency>EUR|NOK|USD) "
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)$"
)


def read_text_file(file_path: str | Path) -> tuple[str, str]:
    """Read a UTF-8 or ISO-8859-1 file and return its text and encoding."""
    path = Path(file_path)

    try:
        return path.read_text(encoding="utf-8"), "utf-8"
    except UnicodeDecodeError:
        return path.read_text(encoding="iso-8859-1"), "iso-8859-1"


def parse_log(file_text: str) -> list[dict]:
    """Parse valid transaction lines and ignore non-matching noise lines."""
```

```
    transactions = []

    for line in file_text.splitlines():
        match = LOG_PATTERN.fullmatch(line.strip())
        if match is None:
            continue

        record = match.groupdict()
        record["price"] = float(record["price"])
        transactions.append(record)

    return transactions


example_text = """[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS
--- PRINTER PAPER LOW WARNING ---"""
print(parse_log(example_text))
```

```
[{'timestamp': '2026-10-15 08:30:00', 'register_id': 'POS_01', 'item': 'Latte', 'pri
ce': 4.5, 'currency': 'EUR', 'status': 'SUCCESS'}]
```

The demonstration returns one dictionary. The warning line is not included because it does not match the complete transaction format.

Task 2 - Recursive utility functions

The following directory function uses explicit recursion as required. It does not use `glob`, `rglob`, or `os.walk`. The configuration function recursively handles any combination of dictionaries and lists and collects registers where `active` is exactly `True`.

In [3]:
```
def find_log_files(root_path: str | Path, extension: str = ".log") -> list[Path]:
    '''Recursively return all files with the requested extension.'''
    root = Path(root_path)
    found_files: list[Path] = []

    for item in sorted(root.iterdir()):
        if item.is_dir():
            found_files.extend(find_log_files(item, extension))
        elif item.is_file() and item.suffix.lower() == extension.lower():
            found_files.append(item)

    return found_files


def find_active_register_ids(value: object) -> set[str]:
    '''Recursively collect active register IDs from dictionaries and lists.'''
    active_ids: set[str] = set()

    if isinstance(value, dict):
        register_id = value.get("id")
        if value.get("active") is True and isinstance(register_id, str):
            active_ids.add(register_id)
```

```
        for nested_value in value.values():
            active_ids.update(find_active_register_ids(nested_value))

    elif isinstance(value, list):
        for nested_value in value:
            active_ids.update(find_active_register_ids(nested_value))

    return active_ids


with CONFIG_PATH.open(encoding="utf-8") as config_file:
    store_config = json.load(config_file)

log_files = find_log_files(DATA_DIR)
active_registers = find_active_register_ids(store_config)

print(f"Log files found: {len(log_files)}")
print(f"Active registers: {sorted(active_registers)}")
```

```
Log files found: 9
Active registers: ['POS_01', 'POS_02', 'POS_03']
```

Task 3 - Cleaning and analysis with NumPy

An outlier is detected using the required rule $|x - \mu| > 3\sigma$. Detected values are replaced with the median of the values that were not classified as outliers. Refund ratios are calculated element by element, with 10^{-6} added to the denominator to prevent division by zero.

In [4]:

```
def remove_price_outliers(price_data) -> np.ndarray:
    '''Replace values more than three standard deviations from the mean.'''
    prices = np.asarray(price_data, dtype=float).reshape(-1)

    if prices.size == 0:
        return prices.copy()

    mean_price = np.mean(prices)
    standard_deviation = np.std(prices)
    outlier_mask = np.abs(prices - mean_price) > 3 * standard_deviation
    cleaned_prices = prices.copy()

    if np.any(outlier_mask):
        valid_prices = prices[~outlier_mask]
        replacement = np.median(valid_prices)
        cleaned_prices[outlier_mask] = replacement

    return cleaned_prices


def compute_refund_ratio(
    sales_count: np.ndarray,
    refund_count: np.ndarray,
) -> np.ndarray:
    '''Return element-wise refund counts divided by successful sales counts.'''
    sales = np.asarray(sales_count, dtype=float)
```

```
    refunds = np.asarray(refund_count, dtype=float)
    return refunds / (sales + 1e-6)


price_test = np.array([3.0] * 20 + [999.0])
cleaned_test = remove_price_outliers(price_test)
print("Outlier before cleaning:", price_test[-1])
print("Value after cleaning:", cleaned_test[-1])
```

```
Outlier before cleaning: 999.0
Value after cleaning: 3.0
```

Task 4 - Data analysis with pandas

The first function builds a chronologically sorted DataFrame. The next three functions create hourly revenue, the status pivot table, and refund ratios per item. Successful transactions are used as the sales count in the denominator of the refund ratio.

In [5]:

```
def build_transaction_dataframe(records: list[dict]) -> pd.DataFrame:
    '''Create a timestamp-indexed, chronologically sorted DataFrame.'''
    dataframe = pd.DataFrame(records)

    if dataframe.empty:
        return pd.DataFrame(
            columns=["register_id", "item", "price", "currency", "status"],
            index=pd.DatetimeIndex([], name="timestamp"),
        )

    dataframe["timestamp"] = pd.to_datetime(dataframe["timestamp"])
    return dataframe.set_index("timestamp").sort_index()


def calculate_hourly_revenue(dataframe: pd.DataFrame) -> pd.DataFrame:
    '''Calculate total transaction price per item in one-hour windows.'''
    return (
        dataframe.groupby("item")["price"]
        .resample("1h")
        .sum()
        .rename("total_revenue")
        .reset_index()
    )


def create_status_pivot(dataframe: pd.DataFrame) -> pd.DataFrame:
    '''Return total transaction price per item and status category.'''
    pivot = pd.pivot_table(
        dataframe,
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0.0,
    )
```

```
    required_statuses = ["SUCCESS", "REFUND", "CANCELLED"]
    return pivot.reindex(columns=required_statuses, fill_value=0.0)


def calculate_item_refund_ratios(dataframe: pd.DataFrame) -> pd.DataFrame:
    '''Calculate successful-sale counts, refund counts, and ratios per item.'''
    items = sorted(dataframe["item"].unique())
    sales_count = np.array([
        ((dataframe["item"] == item) & (dataframe["status"] == "SUCCESS")).sum()
        for item in items
    ])
    refund_count = np.array([
        ((dataframe["item"] == item) & (dataframe["status"] == "REFUND")).sum()
        for item in items
    ])
    ratios = compute_refund_ratio(sales_count, refund_count)

    return pd.DataFrame({
        "item": items,
        "sales_count": sales_count,
        "refund_count": refund_count,
        "refund_ratio": ratios,
    })
```

Task 5 - Complete workflow

The workflow connects all previous functions. It keeps transactions only from active registers, removes outliers before the NOK correction, multiplies NOK prices by 10, and saves all four required output files.

In [6]:
```
def run_sales_pipeline(data_directory: str | Path = "data") -> dict:
    '''Run the complete cleaning and reporting pipeline.'''
    data_path = Path(data_directory)
    config_path = data_path / "store_config.json"

    with config_path.open(encoding="utf-8") as config_file:
        config = json.load(config_file)

    active_ids = find_active_register_ids(config)
    files = find_log_files(data_path)
    records: list[dict] = []
    encoding_counts: dict[str, int] = {}

    for log_path in files:
        text, encoding = read_text_file(log_path)
        encoding_counts[encoding] = encoding_counts.get(encoding, 0) + 1

        for record in parse_log(text):
            if record["register_id"] in active_ids:
                records.append(record)

    transactions = build_transaction_dataframe(records)
```

```
    if transactions.empty:
        raise ValueError("No transactions were found for active registers.")

    transactions["price"] = remove_price_outliers(transactions["price"])
    nok_mask = transactions["currency"].eq("NOK")
    transactions.loc[nok_mask, "price"] *= 10.0
    transactions["price"] = transactions["price"].round(2)

    hourly_revenue = calculate_hourly_revenue(transactions)
    status_pivot = create_status_pivot(transactions)
    refund_ratios = calculate_item_refund_ratios(transactions)

    transactions.reset_index().to_csv(
        "cleaned_transactions.csv", index=False
    )

    hourly_records = hourly_revenue.copy()
    hourly_records["timestamp"] = hourly_records["timestamp"].dt.strftime(
        "%Y-%m-%dT%H:%M:%S"
    )
    Path("hourly_revenue.json").write_text(
        json.dumps(hourly_records.to_dict(orient="records"), indent=2),
        encoding="utf-8",
    )

    status_pivot.to_csv("status_pivot_table.csv")
    refund_ratios.to_csv("item_refund_ratios.csv", index=False)

    return {
        "log_files": files,
        "active_registers": active_ids,
        "encoding_counts": encoding_counts,
        "transactions": transactions,
        "hourly_revenue": hourly_revenue,
        "status_pivot": status_pivot,
        "refund_ratios": refund_ratios,
    }


results = run_sales_pipeline(DATA_DIR)

print("Pipeline completed successfully.")
print("Log files:", len(results["log_files"]))
print("Active registers:", sorted(results["active_registers"]))
print("Transactions retained:", len(results["transactions"]))
print("Encodings reported:", results["encoding_counts"])
```

```
Pipeline completed successfully.
Log files: 9
Active registers: ['POS_01', 'POS_02', 'POS_03']
Transactions retained: 720
Encodings reported: {'utf-8': 9}
```

In [7]:

```
print("Cleaned transactions:")
display(results["transactions"].head())
```

```
print("Hourly revenue:")
display(results["hourly_revenue"].head(12))

print("Status pivot table:")
display(results["status_pivot"])

print("Refund ratios:")
display(results["refund_ratios"])
```

```
Cleaned transactions:
```

timestamp	register_id	item	price	currency	status
2026-10-15 07:00:19	POS_03	Cappuccino	4.68	USD	CANCELLED
2026-10-15 07:01:10	POS_01	Latte	4.48	USD	SUCCESS
2026-10-15 07:01:10	POS_02	Sandwich	7.39	USD	SUCCESS
2026-10-15 07:01:13	POS_02	Espresso	3.08	USD	SUCCESS
2026-10-15 07:01:23	POS_03	Espresso	31.50	NOK	SUCCESS

```
Hourly revenue:
```

	item	timestamp	total_revenue
0	Cappuccino	2026-10-15 07:00:00	836.90
1	Cappuccino	2026-10-15 08:00:00	1094.57
2	Cappuccino	2026-10-15 09:00:00	50.11
3	Croissant	2026-10-15 07:00:00	967.99
4	Croissant	2026-10-15 08:00:00	597.57
5	Croissant	2026-10-15 09:00:00	153.30
6	Espresso	2026-10-15 07:00:00	572.96
7	Espresso	2026-10-15 08:00:00	453.15
8	Espresso	2026-10-15 09:00:00	95.12
9	Latte	2026-10-15 07:00:00	1273.23
10	Latte	2026-10-15 08:00:00	885.30
11	Latte	2026-10-15 09:00:00	143.07

```
Status pivot table:
```

status	SUCCESS	REFUND	CANCELLED
item			
Cappuccino	1319.04	298.87	363.67
Croissant	1118.21	407.04	193.61
Espresso	827.49	220.85	72.89
Latte	1578.20	358.08	365.32
Muffin	1041.39	344.14	279.52
Sandwich	2112.21	474.29	1101.77

Refund ratios:

	item	sales_count	refund_count	refund_ratio
0	Cappuccino	75	17	0.226667
1	Croissant	76	19	0.250000
2	Espresso	64	21	0.328125
3	Latte	92	24	0.260870
4	Muffin	82	19	0.231707
5	Sandwich	86	19	0.220930

In [8]:
```
required_outputs = [
    Path("cleaned_transactions.csv"),
    Path("hourly_revenue.json"),
    Path("status_pivot_table.csv"),
    Path("item_refund_ratios.csv"),
]

for output_file in required_outputs:
    print(f"{output_file.name}: exists={output_file.exists()}, size={output_file.st
```

```
cleaned_transactions.csv: exists=True, size=38690 bytes
hourly_revenue.json: exists=True, size=1961 bytes
status_pivot_table.csv: exists=True, size=231 bytes
item_refund_ratios.csv: exists=True, size=255 bytes
```

In [9]:
```
transactions = results["transactions"]

assert transactions.index.is_monotonic_increasing
assert set(transactions["register_id"]).issubset(results["active_registers"])
assert not np.isclose(transactions["price"].to_numpy(), 999.0).any()
assert all(output_file.exists() for output_file in required_outputs)
assert set(results["status_pivot"].columns) == {
    "SUCCESS", "REFUND", "CANCELLED"
}

print("All validation checks passed.")
```

```
All validation checks passed.
```

Discussion

The workflow successfully separates valid transactions from system messages, filters out inactive registers, and produces chronologically ordered data. The extreme `999.00` hardware-glitch values are detected using the required three-standard-deviation rule and replaced before the currency correction is performed. NOK prices are then multiplied by 10 as specified.

The pivot table makes it possible to compare transaction value across `SUCCESS`, `REFUND`, and `CANCELLED` categories. The refund-ratio report instead uses counts, comparing the number of refunds with the number of successful sales for each item. The hourly report gives one-hour totals for every item.

One limitation is that ISO-8859-1 files containing only ASCII characters are also valid UTF-8. In such cases the reader correctly reads the content but cannot prove which of the two encodings originally produced it. If a Latin-1 file contains a non-UTF-8 character, the fallback is used automatically.

Use of generative AI

Generative AI was used as a support tool during the development of this assignment. It helped us interpret the requirements, structure the functions, and understand concepts such as recursion, regular expressions, NumPy outlier detection, and pandas resampling. It was also used to troubleshoot the Python environment and package installation.

We generated the input data and ran the notebook locally. We checked the program against the assignment specification, verified the generated output files, and used validation checks to confirm that the transactions were sorted, inactive registers were excluded, price outliers were removed, and all required reports were created. AI suggestions were therefore tested and reviewed before being included in the final submission.